

Devoir 1 IFT3335

Simon Pino-Buisson

7 avril 2026

Remise le 24 avril 2026 à 23:59

Introduction

Bienvenus au devoir 2! Dans ce devoir, nous allons explorer la tâche de classification d'image à l'aide de modèles d'apprentissage automatique.

Qu'est-ce que la classification d'image et la vision par ordinateur

Les images digitales sont un regroupement de valeurs de luminosité organisées dans l'espace appelées *pixels*. La résolution d'une image dépend du nombre de pixels. Dans une image en couleur, chaque pixel contient la valeur de luminosité pour 3 couleur (ou *canaux*) généralement comprise entre 0 et 255 appelés RGB (pour red, green and blue). Le RGB couleur noir est (0,0,0) et le blanc est (255,255,255). Tout ceci veut dire qu'une image digitale est simplement une matrice (ou un tenseur pour une image en couleur)! Comme les ordinateurs sont bons avec celles-ci, on a pu faire plusieurs avancées dans le domaine de la vision par ordinateur. Les modèles d'apprentissage automatique forment maintenant une grande partie du sujet pour plusieurs types de tâches.

Ici, nous allons nous concentrer sur la tâche classification d'images qui consiste à associer une étiquette à une image selon le contenu de celle-ci. Les applications de la classification d'images sont nombreuses allant de l'étiquetage automatique d'image, à l'analyse d'images médicales (détection de maladies), au contrôle de qualité ou la détection d'exoplanètes. Il est aussi important de considérer les implications éthiques qui viennent avec cette tâche. Par exemple, la provenance même du jeu de données est importante: il est important d'obtenir le consentement de ceux et celles qui y apparaissent ou qui l'ont créé. Les modèles d'apprentissage automatique souffrent souvent de biais (intentionnels ou non) et la classification d'image n'en est pas exemptée. Finalement, l'utilisation de cette technologie peut évidemment être un problème grave pour plusieurs raisons (non-respect de la vie privée, application discriminatoires, etc...). "*With great power comes great responsibility*" comme dirait oncle Ben. Voici deux articles qui explorent le sujet:

1. Implications éthiques de l'utilisation de modèle d'IA sur l'imagerie médicale
2. Implications éthiques de la vision par ordinateur

Maintenant, avec la matière vue en classe, vous devriez avoir toute l'information nécessaire pour faire le devoir, mais si vous voulez plus d'information, des tonnes de ressources sont disponibles en ligne!

Questions d'approche

Ci-dessous se trouvent quelques questions qui vont vous faire réfléchir au problème avant de commencer. Celles-ci ne sont pas à remettre et ne seront pas corrigées non plus. Par contre, répondre à ces questions (ou au moins y réfléchir) va aider au développement et à votre compréhension de la matière.

1. À quoi sert la fonction d'activation? Est-ce qu'elle est utile pour un MLP à une seule couche? Pourquoi?
2. Pourquoi plusieurs couches? Quels sont les avantages et les inconvénients?

3. Comment peut-on représenter une image de façon utile pour un MLP? Est-ce qu'on doit traiter les données avant de les utiliser pour l'entraînement?
4. Quelles informations est-ce qu'un MLP capture/représente? Quelles informations ne peuvent pas être représentées? Comment est-ce que cela aide/nuît à la tâche de classification d'images?
5. Quelle(s) fonction(s) d'activation a(ont) du sens pour la classification d'image avec un MLP? Pourquoi?
6. Quelle(s) fonction(s) de perte a(ont) du sens pour la classification d'images? Quels sont les différentes conditions?
7. Pour un MLP de 2 couches cachées de tailles h_1 et h_2 , quels sont les avantages/désavantages de $h_1 > h_2$? De $h_1 < h_2$? De $h_1 = h_2$?
8. Pour une image de HxL, combien de paramètres est-ce qu'il y a dans la première couche du réseau? Comment est-ce que l'ajout de canaux (comme RGB) affecte le nombre de paramètres? Comment est-ce qu'on peut les initialiser? Comment est-ce qu'il ne faudrait PAS les initialiser?
9. Quelle métrique de performance est utile ici? Dans quels cas est-ce que la précision serait une mauvaise métrique? Qu'est-ce qu'on pourrait utiliser au lieu?
10. L'augmentation de données est une technique qui consiste à appliquer des modifications aux données existante (telles que des rotations, translations, ajout de bruit, etc...) dans le but d'améliorer l'apprentissage et la généralisation. Quelles augmentations seraient utile pour notre MLP de classification d'images? Quelles transformations nuiraient au modèle?

1 Le devoir

Le *zip* du devoir contient trois fichiers. Voici leurs explications:

1. Part1.py : le fichier d'implémentation des opérations élémentaire en apprentissage automatique pour la partie 1. Ce fichier sera évalué.
2. MLP.py : le fichier d'implémentation du MLP qui sera utilisé dans la partie 2. Ce fichié sera évalué (question 1.1.5).
3. experiments.ipynb : ce fichier contient les étapes pour compléter les expérimentations. Celui-ci ne sert que d'inspiration et n'est pas nécessairement parfait pour l'exécution de toutes les expériences de la partie 2. Vous pouvez le modifier comme bon vous semble, car il ne sera pas évalué. Cependant, sachez qu'il sera remis et vérifié pour plagiat ou utilisation abusive de LLMs génératifs.

Ne pas changer les noms de fichier, les noms de fonctions et le type des objets qu'elles retournent :)

1.1 Partie 1 (50%): Implémrntation des composantes de réseaux de neurones

Cette partie vise à donner de l'expérience dans l'implémentation des différentes composantes élémentaires des réseaux de neurones. Implémenter les différentes fonctions dans les fichiers *Part1.py* et *MLP.py*. La partie 1 sera évaluée automatiquement, alors faites attention de ne pas changer les noms de fonctions et de retourner les bons objets. N'utilisez pas PyTorch pour cette partie.

1.1.1 Implémentation d'un neurone (5%)

Dans le fichier *Part1.py*, implémentez la classe *Neuron* pour pouvoir prendre un vecteur x en entrée et appliquer les vecteurs de poids et de biais w et b , ainsi que la fonction d'activation passée en paramètre tel que vu en classe. N'utilisez pas PyTorch pour cette partie.

1.1.2 Implémentation d'une couche cachée avec fonction d'activation (10%)

Dans le fichier *Part1.py*, implémentez la classe *Layer* pour pouvoir prendre un vecteur x en entrée et appliquer la matrice de poids w , le vecteur de biais b et la fonction d'activation tel que vu en classe. De plus, implémentez la classe de façon à ce que la structure de l'entrée puisse être modifiée par le paramètre *input_structure*. N'utilisez pas PyTorch pour cette partie.

1.1.3 Implémentation de fonctions d'activations (5%)

Dans le fichier *Part1.py*, implémentez les fonctions d'activation *sigmoid*, *tanh*, *leaky ReLu* et *softmax* telles que vue en classe. N'utilisez pas PyTorch pour cette partie.

1.1.4 Implémentation de fonctions de perte (5%)

Dans le fichier *Part1.py*, implémentez les fonctions de perte *MAE*, *MSE*, *log-cosh*, et *cross-entropy* telles que vues en classe. N'utilisez pas PyTorch pour cette partie.

1.1.5 Implémentation d'un réseau de neurones à plusieurs couches (10%)

Dans le fichier *MLP.py*, implémentez un réseau de neurones de type MLP à l'aide de PyTorch (pour accélérer le processus d'entraînement). Assurez vous de rendre la structure modulaire et dépendante des paramètres du constructeur pour pouvoir faire les expériences de la partie 2. Suivez les instructions des méthodes marquées *TODO*.

1.1.6 Implémentation de l'algorithme d'entraînement (15%)

Dans le fichier *Part1.py*, implémentez la fonction *train* qui s'occupe de faire l'entraînement d'un modèle selon une liste d'hyperparamètres. Suivez l'algorithme vu en classe et les sections marquées *TODO*. L'utilisation de PyTorch est permise ici (ceci accélère grandement le processus d'entraînement).

1.2 Partie 2 (50%): Exploration des effets des hyperparamètres

Cette partie vise à démontrer votre compréhension des réseaux de neurones et de l'apprentissage automatique. Vous devez répondre aux questions suivantes sous forme de rapport.

Excepté pour la première question, vous devez entraîner les 10 modèles décrits au tableau 1 et commenter les comparaisons suggérées. Les exigences sont assez ouvertes. Décrivez brièvement ce qui se passe (en environ 1 à 4 paragraphes... pas de rapports de 50 pages svp), pourquoi un modèle performe mieux qu'un autre, en quoi cette expérience est intéressante, comment les hyperparamètres affectent l'entraînement et la généralisation, est-ce que c'est l'effet attendu?, les avantages/inconvénients... Vous devez également joindre un graph de l'apprentissage qui représente bien l'effet de l'hyperparamètre dont il est question (généralement perte et précision pour train et test, mais pas nécessairement). Si vous jugez que c'est pertinent, vous pouvez ajouter des expériences aux comparaisons ou légèrement changer les valeurs d'hyperparamètres. Cependant il est TRÈS important de comparer des modèles différant seulement d'un hyperparamètre.

1.2.1 Comprendre la matrice de poids (5%)

1. Soit un réseau neuronal de type MLP avec une couche l de 10 neurones et une couche $l + 1$ de 20 neurones. Les deux couches sont complètement connectées, mais on veut que les neurones de l soient connectés tel que vu dans la figure 1. C'est à dire: les 5 premiers neurones de l sont connectés aux neurones 1, 3, 5, ... de $l + 1$ et les 5 derniers neurones de l sont connectés aux neurones 2, 4, 6, ... de $l + 1$. Décrivez à quoi ressemble la matrice de poids entre les couches l et $l + 1$.
2. Combien y a-t-il de paramètres (incluant les biais) dans un MLP complètement connecté qui prend en input une image de 28x28 pixels, a deux couches cachées de 64 neurones et une couche output de 10 neurones?

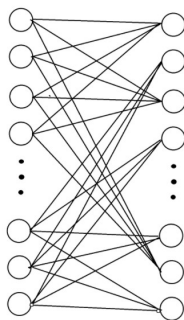


Figure 1: Connexions d'un MLP.

1.2.2 Expérience 1 : Normalisation des données (5%)

Comparez les modèles Baseline (normalisé) et 1 (non normalisé) pour voir l'effet de la normalisation des données.

1.2.3 Expérience 2 : Nombre de couches (5%)

Comparez les modèles Baseline (2 couches cachées) et 2 (1 couche cachée) pour voir l'effet de la profondeur dans un réseau de neurones. Pourquoi avons nous choisi 70 neurones pour l'expérience 2?

1.2.4 Expérience 3 : Taux d'apprentissage (10%)

Comparez les modèles Baseline (taux d'apprentissage de 0.05), 3 (taux d'apprentissage de 0.1) et 4 (taux d'apprentissage de 0.001) pour voir l'effet du taux d'apprentissage.

1.2.5 Expérience 4 : Fonctions de pertes (10%)

Comparez les modèles Baseline (Entropie croisée), 5 (Hinge) et 6 (MSE) pour voir l'effet de différentes fonction de perte. Nous vous avons fourni les fonctions hinge et MSE dans `experiments.ipynb` pour faciliter l'implémentation.

1.2.6 Expérience 5 : Taille des couches cachées (10%)

Comparez les modèles Baseline ($h1 = 64$, $h2 = 64$), 7 ($h1 = 48$, $h2 = 300$) et 8 ($h1 = 300$, $h2 = 48$) pour voir l'effet de la structure du réseau. Pourquoi avons nous choisi les tailles 48 et 300?

1.2.7 Expérience 6 : Taille des minibatches (5%)

Comparez les modèles Baseline (batch size = 64) et 10(batch size = 8) pour voir l'effet de la taille des minibatches.

2 Quoi remettre

Après avoir créé votre compte Gradescope si ce n'est pas déjà fait, aller dans la section devoir 2 et remettez les fichiers *Part1.py*, *MLP.py* et *experiments.ipynb* en entier. Le notebook des expériences ne sera pas noté, mais il sera être consulté pour vérifier le plagiat, l'utilisation des LLMs génératifs ou pour expliquer des erreurs dans le rapport.

Vous devez remettre le rapport de la partie 2 sous forme de pdf appelé *partie2.pdf* sur gradescope.

# d'expérience	Baseline	1	2	3	4	5	6	7	8	9
Normalization des données	Oui	Non	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui
Nombre de couches	2	2	1	2	2	2	2	2	2	2
Taux d'apprentissage	0.05	0.05	0.05	0.1	0.01	0.05	0.05	0.05	0.05	0.05
Fonctions de pertes	CE	CE	CE	CE	CE	Hinge	MSE	CE	CE	CE
Taille de H1	64	64	70	64	64	64	64	48	300	64
Taille de H2	64	64	0	64	64	64	64	300	48	64
Taille de minibatch	64	64	64	64	64	64	64	64	64	2
Epochs	20	20	20	20	20	20	20	20	20	20

Table 1: Hyperparamètres de chaque expérience

3 Correction

La correction se fait automatiquement avec différent test unitaires pour la partie 1. Les tests indiquent ce qui n'a pas fonctionné, mais attention: il n'y a pas de tests cachés pour ce devoir. Passer les tests sur gradescope donne la note de 50% au devoir. La partie 2 sera corrigée à la main et vaut 50%.

4 L'utilisation des LLMs

Les LLMs vont faire partie de notre vie professionnelle. Leur utilisation est inévitable et peut même être utile à l'apprentissage. Par contre, s'ils écrivent le devoir pour vous ça ne sert à rien. L'utilisation d'intelligence artificielle pour la génération de code et de la création de solution est interdite. Le but est d'apprendre et réfléchir aux problèmes. De plus, nous vous rappelons la politique de différence inexplicable entre les notes de quiz/devoir et d'examen. Des mesures disciplinaires seront entreprise en cas de doute d'utilisation inadéquate des IA génératives. Écrivez-nous si vous avez une question à laquelle vous pensez réellement être incapable de répondre:)

Bon succès!